Agent Phantom Recovery

10-backend-schema.md

Production Backend Schema

Version: 1.0

---

1. Purpose

This document defines the logical backend schema for Agent Phantom Recovery.

Unlike the database schema, this document describes:

- Services
- Modules
- Domain Objects
- Internal Contracts
- Event Structures
- Queues
- State Machines
- Execution Models
- Service Communication

This document serves as the bridge between:

```
System Architecture
        ↓
Backend Architecture
        ↓
Database Schema
        ↓
API Specification
```

and acts as the internal blueprint for backend implementation.

---

2. Backend Schema Philosophy

The backend is designed around execution rather than conversation.

Traditional systems:

```
Request
 ↓
Process
 ↓
Response
```

Agent Phantom Recovery:

```
Goal
 ↓
Plan
 ↓
Evidence
 ↓
Reasoning
 ↓
Verification
 ↓
Execution
 ↓
Delivery
```

The backend schema reflects this execution-first design.

---

3. Core Domain Model

The entire backend revolves around several primary entities.

```
User
Project
Repository
Task
Plan
Memory
Tool
Execution
Finding
Patch
Verification
Report
```

Everything else extends these core entities.

---

4. Domain Relationships

```
User
 │
 ├── Projects
 │
 ├── Tasks
 │
 └── Memories

Project
 │
 ├── Repositories
 ├── Tasks
 └── Reports

Task
 │
 ├── Plan
 ├── Evidence
 ├── Executions
 ├── Findings
 ├── Verifications
 └── Reports
```

This relationship graph drives most backend interactions.

---

5. User Domain

User Object

```
User {
    id
    email
    username
    role
    status
    createdAt
    updatedAt
}
```

Responsibilities

Stores:

- identity
- ownership
- permissions
- preferences

---

6. Project Domain

Project Object

```
Project {
    id
    ownerId
    name
    description
    visibility
    status
    createdAt
}
```

Purpose

Represents a long-term workspace.

Projects group:

- repositories
- tasks
- memories
- reports

---

7. Repository Domain

Repository Object

```
Repository {
    id
    projectId
    source
    branch
```

```
    status
    metadata
}
```

Repository Metadata

Contains:

```
RepositoryMetadata {
    language
    framework
    dependencies
    size
    fileCount
}
```

---

8. Task Domain

Task Object

The most important entity.

```
Task {
    id
    projectId
    goal
    status
    priority
    currentPlanId
    createdAt
    updatedAt
}
```

Task States

```
CREATED
PLANNING
COLLECTING_EVIDENCE
ANALYZING
EXECUTING
VERIFYING
COMPLETED
```

```
FAILED
BLOCKED
```

Why Task-Centric?

Everything the platform does belongs to a task.

Tasks become the backbone of:

- orchestration
- observability
- recovery

---

9. Plan Domain

Plan Object

```
Plan {
    id
    taskId
    version
    status
    summary
}
```

Plan Step

```
PlanStep {
    id
    planId
    order
    description
    tool
    status
}
```

Example

```
1. Run Tests
2. Inspect Logs
```

```
3. Analyze Auth Module
4. Generate Patch
5. Verify Fix
```

---

10. Evidence Domain

Evidence is the foundation of reasoning.

Evidence Object

```
Evidence {
    id
    taskId
    source
    type
    content
    confidence
}
```

Evidence Types

```
LOG
TEST_RESULT
FILE
CODE
TERMINAL_OUTPUT
SEARCH_RESULT
BROWSER_OBSERVATION
MEMORY
```

Reasoning Rule

No major conclusion should exist without evidence.

---

11. Finding Domain

Finding Object

```
Finding {
    id
    taskId
    evidenceIds[]
    summary
    severity
    confidence
}
```

Examples

```
Authentication Bug
Memory Leak
Configuration Issue
Dependency Conflict
```

Purpose

Represents an analyzed issue supported by evidence.

---

12. Patch Domain

Patch Object

```
Patch {
    id
    taskId
    findingId
    summary
    filesModified[]
    status
}
```

Patch States

```
DRAFT
GENERATED
TESTED
VERIFIED
REJECTED
```

Why Separate?

A finding may produce:

- one patch
- multiple patches
- no patch

The schema must support all scenarios.

---

13. Verification Domain

Verification Object

```
Verification {
    id
    taskId
    patchId
    status
    confidence
}
```

Verification Checks

```
VerificationCheck {
    id
    verificationId
    type
    result
}
```

Examples

```
TESTS_PASSED
REGRESSION_CHECK
PATCH_REVIEW
VERIFIER_APPROVAL
```

---

14. Report Domain

Report Object

```
Report {
    id
    taskId
    type
    status
}
```

Report Types

```
TASK_SUMMARY
BUG_REPORT
PATCH_REPORT
VERIFICATION_REPORT
FINAL_REPORT
```

---

15. Memory Schema

Memory is one of the most important systems.

Base Memory Object

```
Memory {
    id
    projectId
    type
    content
    importance
```

```
    createdAt
}
```

Working Memory

```
WorkingMemory {
    taskId
    activeContext
}
```

Session Memory

```
SessionMemory {
    sessionId
    interactions
}
```

Project Memory

```
ProjectMemory {
    repositoryKnowledge
    architectureKnowledge
}
```

Experience Memory

```
ExperienceMemory {
    strategy
    outcome
    usefulnessScore
}
```

---

16. Agent Schema

Agent Object

```
Agent {
    id
    type
    status
}
```

Agent Types

```
PLANNER
REASONER
VERIFIER
```

Purpose

Tracks execution identity.

Useful for:

- observability
- metrics
- debugging

---

17. Tool Schema

Tool Object

```
Tool {
    id
    name
    category
    status
}
```

Categories

```
TERMINAL
BROWSER
FILESYSTEM
SEARCH
GIT
CODE_RUNNER
```

Tool Execution Object

```
ToolExecution {
    id
    taskId
    toolId
    status
    startedAt
    finishedAt
}
```

---

18. Terminal Schema

Terminal Session

```
TerminalSession {
    id
    taskId
    environmentId
}
```

Command

```
Command {
    id
    sessionId
    command
    output
}
```

Antigravity Integration

Maps directly to:

```
Terminal Workspace
```

---

19. Browser Schema

Browser Session

```
BrowserSession {
    id
    taskId
    url
}
```

Browser Observation

```
BrowserObservation {
    id
    sessionId
    observation
}
```

Antigravity Integration

Maps directly to:

```
Browser Workspace
```

---

20. Execution Schema

Execution Object

```
Execution {
    id
    taskId
```

```
    action
    status
}
```

Statuses

```
QUEUED
RUNNING
COMPLETED
FAILED
```

Purpose

Tracks every action performed.

---

21. Queue Schema

Queue Item

```
QueueItem {
    id
    taskId
    type
    priority
}
```

Queue Types

```
PLANNING
ANALYSIS
EXECUTION
VERIFICATION
REPORTING
```

---

22. Event Schema

Event Object

```
Event {
    id
    type
    source
    timestamp
}
```

Common Events

```
TaskCreated
TaskStarted
TaskCompleted

ToolStarted
ToolFinished

VerificationPassed
VerificationFailed
```

---

23. State Machine Schema

Task State

```
TaskState {
    current
    previous
    enteredAt
}
```

Purpose

Supports:

- recovery
- observability
- checkpoints

24. Checkpoint Schema

Checkpoint Object

```
Checkpoint {
    id
    taskId
    state
    snapshot
}
```

Why Important?

Long-horizon tasks must survive:

- crashes
- restarts
- failures

---

25. Service Communication Schema

Services communicate through contracts.

Planner Output

```
PlannerResult {
    taskId
    plan
    nextActions
}
```

Reasoner Output

```
ReasonerResult {
    findings
    confidence
}
```

Verifier Output

```
VerifierResult {
    approved
    comments
}
```

---

26. Repository Analysis Schema

Repository Index

```
RepositoryIndex {
    repositoryId
    files
    symbols
}
```

Call Graph

```
CallGraph {
    nodes
    edges
}
```

Dependency Graph

```
DependencyGraph {
    packages
    relations
}
```

---

27. Observability Schema

Trace Object

```
Trace {
    id
    taskId
    events[]
}
```

Metric Object

```
Metric {
    name
    value
    timestamp
}
```

Purpose

Enables:

- monitoring
- debugging
- analytics

---

28. Antigravity Workspace Mapping

Backend entities map directly to UI components.

Browser Workspace

```
BrowserSession
BrowserObservation
```

Source Code Workspace

```
Repository
Finding
Patch
```

Terminal Workspace

```
TerminalSession
Command
Execution
```

Chat Workspace

```
Task
Plan
Memory
```

Timeline Workspace

```
Event
Trace
```

Memory Workspace

```
Memory
ProjectMemory
ExperienceMemory
```

---

29. Schema Design Principles

The backend schema follows:

Separation of Concerns

Reasoning is separate from execution.

Event-Driven Architecture

Everything important produces events.

Traceability

Every action can be reconstructed.

Long-Horizon Support

State survives interruptions.

Extensibility

Future modules can be added without redesign.

---

30. Final Backend Schema Summary

The backend schema is centered around:

```
User
 ↓
Project
 ↓
Repository
 ↓
Task
 ↓
Plan
 ↓
Evidence
 ↓
Finding
 ↓
Patch
 ↓
Verification
 ↓
Report
```

supported by:

```
Memory
+
Agents
+
Tools
+
Events
+
Executions
+
Checkpoints
```

This schema forms the logical backbone of Agent Phantom Recovery and serves as the foundation for the upcoming database schema, API specification, service contracts, and implementation modules.